
1 Teste automático remoto de servidores GS

O presente guia contém os procedimentos para execução remota da aplicação ‘player’ na máquina ‘tejo’ para teste dos servidores GS desenvolvidos pelos alunos.

1.1 Introdução

No âmbito do projecto de comunicação usando a interface sockets torna-se necessário testar as aplicações desenvolvidas pelos alunos em comunicação com aplicações que cumpram o protocolo especificado.

Tal é a finalidade do servidor concorrente GS em execução na máquina ‘tejo’, o qual permite testar as aplicações ‘player’ desenvolvidas pelos alunos.

Com a finalidade inversa de testar as aplicações GS desenvolvidas pelos alunos, existe em execução na máquina ‘tejo’ um servidor TCP concorrente (no porto 59000) que invoca localmente a aplicação ‘player’ em resposta a um acesso TCP por **netcat**, originado num computador dos alunos, através do qual se especificam os parâmetros de execução remota da aplicação ‘player’.

Assim é possível executar em simultâneo na máquina ‘tejo’ instâncias da aplicação ‘player’ que enviam mensagens a servidores GS alvo em execução na rede pública ou na rede do Técnico.

Ambas as modalidades de teste acima referidas permitem aos alunos suportar tanto o desenvolvimento das suas aplicações como a componente de autoavaliação.

As aplicações ‘player’ que serão executadas remotamente na máquina ‘tejo’ vão ler as sequências de comandos a executar a partir de

scripts predefinidos, escolhidos pelos alunos para testar os seus servidores GS. As instâncias da aplicação ‘player’ invocadas remotamente de forma concorrente para execução no tejo mantêm separação de dados não havendo qualquer interferência entre aplicações ‘player’ que estejam em execução simultânea.

A figura que se segue ilustra uma sequência de teste típica.

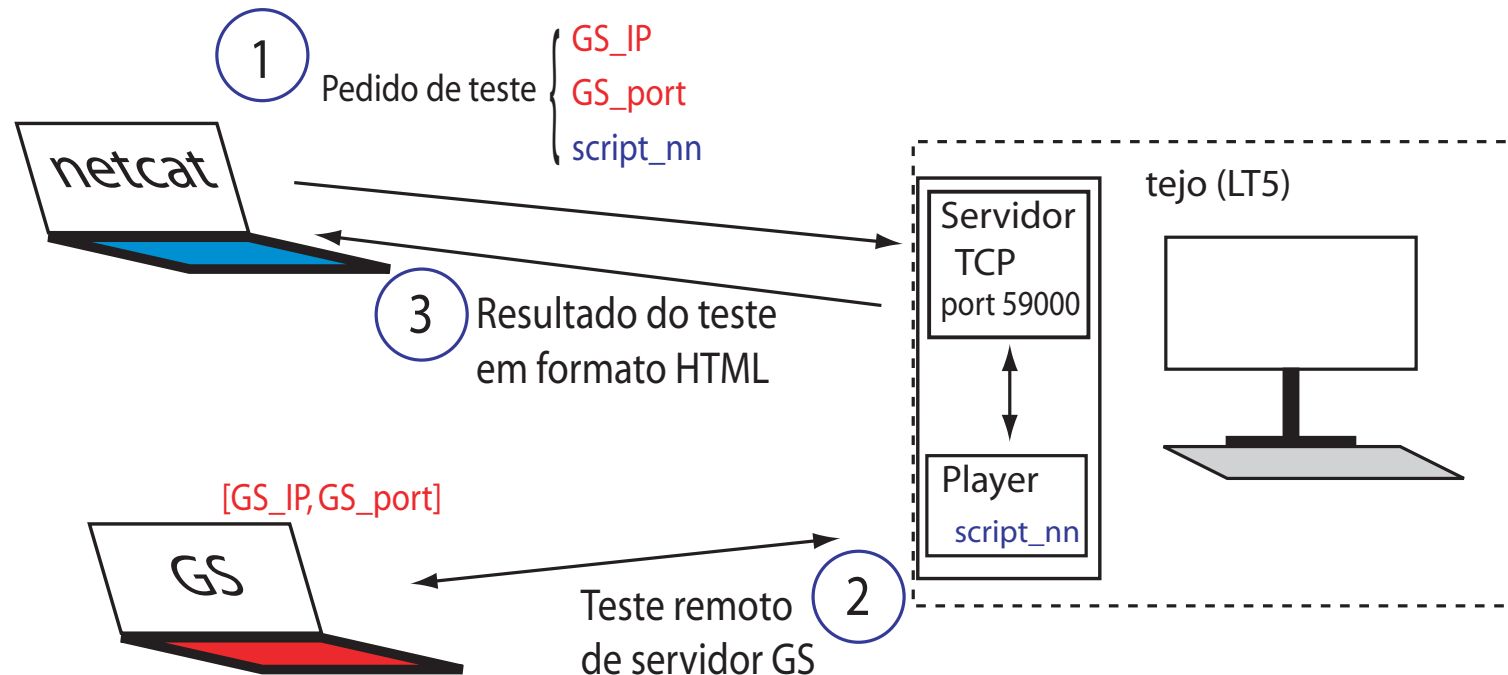


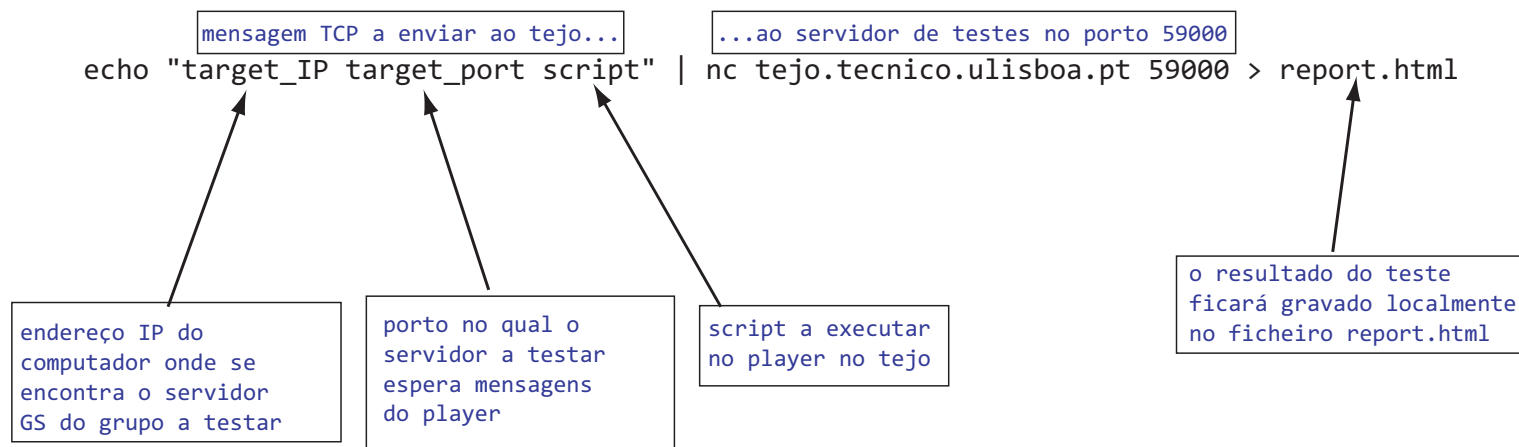
Figure 1: Teste remoto de servidores GS

Para executar um teste subordinado a um dado *script*, os alunos têm de executar a seguinte linha de comando no Linux:

```
echo "target_IP target_port script" | nc tejo.tecnico.ulisboa.pt 59000 > report.html
```

na qual, ‘target_IP’ é o endereço IP da máquina onde executam o seu servidor GS, ‘target_port’ é o porto da sua aplicação GS na referida máquina e ‘script’ é um inteiro com um ou dois dígitos que indica o número do *script* de teste que querem correr (ver *scripts* publicados na

página da disciplina), como se ilustra no seguinte diagrama:



Não se deve invocar o **nc**, para depois enviar a mensagem ‘target_IP target_port script’. Porque a invocação do **nc** inicia de imediato a ligação TCP com o servidor de testes. Como o tempo de composição da mensagem é algo demorado, o servidor abandona a ligação por *timeout* antes de receber a mesma. Por isso se deve sempre compor integralmente todo o comando de activação tal como ilustrado acima. Na mensagem com o formato apresentado acima, o servidor no tejo espera encontrar o caracter ‘\n’ logo a seguir ao número do *script*, o qual é inserido pelo próprio comando ‘echo’ nas instalações que serviram para elaborar este guia. Caso o comando ‘echo’ não insira o ‘\n’ deve ser usado um procedimento alternativo que o faça.

Os espaços entre campos podem ser em qualquer número desde que o comprimento total da mensagem (incluindo ‘\n’) não ultrapasse os 25 bytes. Não são aceites pedidos com os endereços IP público ou privado do tejo, ou igual a 127.*.*.

No final da execução, o servidor envia um ficheiro HTML com o resultado da referida execução obtido pela aplicação ‘player’ cuja execução se solicitou. Esse ficheiro pode ser convertido em formato pdf para ser entregue no contexto da auto-avaliação do projecto.

O servidor GS alvo a testar pode ser executado no *sigma* cujo endereço IPv4 pode ser obtido executando o comando *hostname -i*. Note-se que o *sigma* pode ser endereçado por vários IPs. Para testarem o servidor GS no seu domicílio, os alunos devem proceder à configuração

de *port forwarding* nos seus *routers*.

Como exemplo, considere-se o servidor GS a testar no porto 58050 do *sigma* com IP=193.136.128.103 para execução do *script* número 6. A linha a executar numa máquina com Linux (podendo ser ela o próprio *sigma*) será:

```
echo "193.136.128.103 58050 6" | nc tejo.tecnico.ulisboa.pt 59000 > report_6.html
```

Ficando o relatório da execução guardado no ficheiro report_6.html na máquina na qual se executa o comando **nc**.

Apresenta-se de seguida uma descrição sumária dos testes disponíveis:

1.2 Testes simples de activação de jogos

O servidor GS deve activar jogos quando recebe as mensagens SNG ou DBG. Em ambos os casos verifica-se o correcto funcionamento do servidor pela recuperação do código adoptado no servidor. Quer por geração aleatória quando recebe SNG, quer por aceitação do código enviado pelo *player* na mensagem DBG. Em resposta à mensagem QUT, o servidor deve retornar ao *player* o código que adoptou no início do jogo.

Os *scripts* numerados de 01 a 04 verificam o código secreto que o servidor adoptou no início do jogo.

1.3 Testes de temporização de jogos - UDP

O servidor GS deve dar como terminado um jogo cujo tempo de actividade ultrapassou o limite especificado inicialmente.

O *script* 05 efectua um teste de temporização usando o comando *quit*. Primeiramente o player faz arrancar um jogo com duração de 20 segundos e de imediato desiste do mesmo, devendo receber do servidor a mensagem de confirmação com revelação do código secreto. De seguida efectua outro teste também com duração de 20 segundos, suspende a actividade durante 30 segundos e desiste do jogo. Neste caso está a desistir de um jogo já inactivo, devendo portanto receber como resposta a mensagem RQT NOK\n.

O *script* 06 efectua um teste de temporização usando o comando *try*. Primeiramente o player faz arrancar um jogo em modo *debug*. Depois efectua três tentativas, com intervalos de espera entre elas, por forma a que a última ocorra fora do prazo de validade do jogo. O servidor deve responder de forma apropriada.

1.4 Testes de verificação de código

Os *scripts* numerados de 07 a 09 efectuam testes de verificação de código no servidor.

1.5 Testes de finalização de jogos - TCP

O *script* 10 testa a finalização de jogos pelos quatro motivos previstos no projecto (WIN, Timeout, Quit e nt>8) com pedidos de *show_trials* antes e após a finalização dos jogos.

1.6 Testes de consulta de *scores*

O *script* 11 testa o cálculo e a consulta de *scores*. Antes do início dos testes com este *script* deve eliminar-se todo o conteúdo de ambas as directorias que armazenam estados de jogos e *scores*.

A execução de cada *script* é razoavelmente rápida. No entanto, no caso dos scripts contendo comandos *sleep*, o tempo de espera pelo relatório será no mínimo igual à soma dos tempos de espera (que estão em segundos) contidos nos comandos *sleep* do *script*.

1.7 Testes de concorrência

Os testes de concorrência são efectuados invocando remotamente quatro execuções simultâneas do *player* no ‘tejo’.

Essas quatro execuções são comandadas pelos *scripts* 21, 22, 23 e 24. Para tal devem ser preparados 4 terminais Linux com as seguintes linhas de comando:

```
echo "target_IP target_port 21" | nc tejo.tecnico.ulisboa.pt 59000 > report_21.html  
echo "target_IP target_port 22" | nc tejo.tecnico.ulisboa.pt 59000 > report_22.html  
echo "target_IP target_port 23" | nc tejo.tecnico.ulisboa.pt 59000 > report_23.html  
echo "target_IP target_port 24" | nc tejo.tecnico.ulisboa.pt 59000 > report_24.html
```

Cada uma das quatro instâncias do *player* assim invocadas tem acesso ao estado das três restantes e espera pelo arranque das outras três, para que todas se iniciem simultaneamente. Esta funcionalidade restringe-se apenas à execução dos *scripts* 21, 22, 23 e 24. Não obstante esta funcionalidade de sincronização, os alunos devem activar as quatro linhas de comando acima ilustradas com um mínimo de intervalo de tempo entre elas por forma a evitar a ocupação desnecessária de recursos no ‘tejo’.